Wormhole Proofs: Recursive Zero-Knowledge Hash-Based Authorization with #P-Hard Privacy

Christopher Smith and Ethan Cemer

Quantus Labs, USA

Abstract. Post-quantum signature schemes such as ML-DSA impose public keys and signatures an order of magnitude larger than their elliptic-curve counterparts, substantially increasing communication and storage costs. An alternative approach replaces signature verification with *hash-based authorization*: a prover demonstrates knowledge of a secret preimage tied to a public commitment, rather than presenting a signature. This sidesteps lattice-based operations entirely, requiring only hash evaluations and membership proofs.

Building on the wormhole mechanism from EIP-7503, we formalize a zero-knowledge *wormhole proof system* where funds are committed to addresses derived from secret preimages and later claimed by proving knowledge of the preimage in zero knowledge. We construct a two-layer recursive proof architecture using Plonky2 with FRI commitments. To achieve privacy against balance-linking adversaries, we introduce client-side dummy padding and local shuffling at the first aggregation layer, enlarging the effective anonymity set.

We give complete game-based security definitions and proofs for deposit binding, one-time withdrawal, spend-path exclusivity, value conservation, nullifier indistinguishability, and transaction unlinkability. Under a uniform prior over plausible deposit subsets, adversarial identification advantage is bounded by $1/k$ for an anonymity set of size k; we further prove that even computing k is #P-hard. Our implementation using Plonky2 shows 20 ms proving time per authorization with 1.6 ms verification. Two-layer recursive aggregation scales to 256 authorizations with near-constant final verification (approximately 3.7 ms at the largest batch tested). The marginal on-chain cost is only 56 bytes per settled output, over 80× smaller than a single ML-DSA-87 signature.

Keywords: zero-knowledge proofs · post-quantum cryptography · recursive aggregation · wormhole proofs · anonymity sets · subset sum

1 Introduction

The post-quantum transition poses a fundamental dilemma for blockchains. While Shor's algorithm breaks elliptic-curve signatures, the standardized post-quantum replacements such as ML-DSA have signatures and public keys an order of magnitude larger than their classical counterparts. Naively substituting them increases on-chain bandwidth and state growth dramatically, without addressing transaction privacy.

Verifying post-quantum signatures inside a zero-knowledge circuit and recursively aggregating them is one path, but the circuit cost remains substantial. This paper takes a different approach: users prove *knowledge of a secret preimage* corresponding to a public wormhole address rather than proving a signature. This idea builds on Ziggy's hash-preimage signatures and EIP-7503's wormhole concept, but we develop it into a practical post-quantum private settlement system.

E-mail: chris@quantus.com (Christopher Smith), ethan@quantus.com (Ethan Cemer)

Date of this document: 2026-06-12.

We formalize the wormhole proof relation, design a two-layer recursive aggregation protocol with client-side privacy mechanisms, prove six core security properties in the game-based style, and show that computing the effective anonymity set is #P-hard. Our Plonky2 implementation achieves practical performance with only 56 bytes of marginal on-chain cost per settled output.

Our Contributions

- Formalization of the wormhole proof relation and a precise security model for post-quantum private settlement.
- A two-layer recursive aggregation architecture with client-side dummy padding and shuffling that enlarges anonymity sets while remaining efficient.
- Proof that computing the size of the anonymity set is #P-hard, together with game-based security proofs for deposit binding, one-time withdrawal, spend-path exclusivity, value conservation, nullifier indistinguishability, and transaction unlinkability.
- A practical Plonky2 implementation achieving 20 ms per authorization and 56 bytes marginal on-chain cost per settled output, over 80× smaller than an ML-DSA-87 signature, with near-constant recursive verification (under 4 ms for up to 256 authorizations).

Paper Organization

Section 2 reviews background on wormholes, privacy pools, and transparent recursion with FRI. Section 3 defines our security model. After a protocol overview in Section 4 and formal interfaces in Section 5, Section 6 details the recursive construction. Section 7 contains our security definitions and proofs. We report performance in Section 8 and discuss related work in Section 9.

2 Background and Design Goals

2.1 Wormholes and Privacy Pools

EIP-7503 proposes *zero-knowledge wormholes*: deposit to secret-derived addresses, then prove knowledge of the secret to withdraw [KBK+23]. We extend this concept with post-quantum proving and recursive aggregation with dummy padding for privacy.

Related systems include privacy pools and shielded ledgers like Zcash and Tornado Cash [KBK+23, BIN+23, HBH+23, PSS19]. Unlike fully shielded ledgers, deposits remain visible as ordinary transfer events on-chain; privacy comes from proving knowledge of a secret without revealing which deposit funded the withdrawal. The circuit authenticates deposits via inclusion in a consensus-maintained *ZK tree*—a 4-ary sorted Poseidon Merkle tree of transfer events whose root is committed in the block header—rather than via encrypted note commitments or Merkle–Patricia storage proofs. Importantly, no ciphertexts are stored, so a future break in cryptographic assumptions cannot retroactively compromise transaction privacy.

2.2 Transparent Recursion

We use Plonky2, combining PLONK arithmetization with FRI commitments for recursive verification without trusted setup [Pol22, BBHR18]. FRI is based on hash functions rather than elliptic curves, making it plausibly post-quantum secure: known quantum attacks on hash preimage search (Grover) and collision finding (BHT) improve over classical bounds,

but 256-bit outputs retain comfortable margins at our security level. Poseidon2 provides efficient in-circuit hashing over prime fields [GKS23].

Security Parameters: Our instantiation uses the Goldilocks field ($p = 2^{64} - 2^{32} + 1$) with a quadratic extension for 100-bit conjectured soundness. FRI is configured with 28 query rounds, rate 1/8 (blowup factor 2^3), and 16 bits of proof-of-work grinding, yielding $28 \times 3 + 16 = 100$ bits of conjectured security against proof forgery under Plonky2's standard FRI soundness conjecture; provable bounds in the Johnson list-decoding regime are lower [Pol22, BBHR18]. Poseidon2 outputs 4 field elements (256 bits), giving roughly 128-bit classical collision resistance and 256-bit classical preimage resistance. Against a quantum adversary, Grover search reduces preimage work to about 2^{128} operations, while generic quantum collision attacks (e.g., BHT) reduce collision work to about 2^{85} operations—still above our 100-bit soundness target. Hash output length, not the Goldilocks field size, therefore governs post-quantum hash margins.

2.3 Design goals

The protocol aims to satisfy four simultaneous goals:

1. **Amortized Verification:** The marginal on-chain cost of verifying an additional withdrawal should be small.
2. **Deposit Binding:** Every withdrawal must be bound to a real deposit recorded in authenticated chain state.
3. **Replay Prevention:** A deposit should not authorize multiple withdrawals; nullifiers must uniquely bind proofs to deposits.
4. **Meaningful Privacy:** The chain should reveal only the public information needed for settlement: exits, amounts, the asset identifier, shared fee parameters, nullifiers, and a block hash with its block number. Funding accounts, secrets, and ZK tree inclusion proofs should remain private.

3 System, Adversary, and Trust Model

3.1 Roles

- **User / Prover:** Generates a secret, derives a wormhole address, deposits funds, and later produces a zero-knowledge proof to withdraw. First-layer aggregation should be performed locally to preserve privacy.
- **Aggregator:** Combines multiple proofs into a single aggregate proof. This role may be the user, a third-party service, or a block producer. Higher-layer aggregation can be delegated since it operates only on public outputs.
- **Blockchain / Verifier:** Validates the final proof, checks nullifier freshness, and settles withdrawals to the specified exit accounts.

3.2 Adversarial capabilities

We assume a probabilistic polynomial-time adversary that can:

- observe the full chain and all public proof outputs,
- submit arbitrary deposits,

- perform timing analysis, and
- attempt to replay or malform proof submissions.

The adversary cannot:

- break the collision resistance of the hash function,
- break the knowledge soundness of the proof system, or
- violate the chain's block-history availability.

3.3 Trust assumptions

The protocol relies on the following assumptions.

1. **Hash Security:** The Poseidon2 instantiation used by the system remains collision resistant and preimage resistant for the relevant security level [GKS23].
2. **Proof Soundness:** The configured Plonky2 recursion stack remains knowledge sound under its standard assumptions [Pol22, BBHR18].
3. **Authenticated History:** The blockchain can check that the public block hash and block number correspond to a valid block in chain history.
4. **Witness Availability:** Honest users can obtain the ZK tree inclusion proof needed to build a witness for their deposit.

4 Protocol Overview

The protocol consists of five stages.

1. Secret Generation and Deposit: The user samples a secret s and derives a wormhole address

$$\mathsf{WA}(s) = H(H(\mathsf{salt}_{\mathsf{wh}} \parallel s)).$$

Because this address is derived from a hash rather than a public key, no one can spend from it via ordinary signature-based authorization—wormhole addresses can receive funds but never have outgoing transactions. Anyone can deposit to a wormhole address through an ordinary on-chain transfer, creating a record that can later be authenticated inside the circuit. A user may receive funds from their own prior transfers or from third-party payments to wormhole addresses derived from secrets the user controls.

2. Witness Acquisition: To withdraw, the user gathers the deposit record (a transfer event to their wormhole address) and a ZK tree inclusion proof showing that event is included under the block's committed tree root. These form the private witness.

3. Leaf Proof Generation: The user chooses up to two exit outputs, computes a nullifier

$$\mathsf{Null}(s, c) = H(H(\mathsf{salt}_{\mathsf{null}} \parallel s \parallel c)),$$

and produces a leaf proof. The proof reveals the nullifier, the exits, the output amounts, the asset identifier, the fee basis points, the block hash, and the block number, while keeping the secret and ZK tree inclusion proof private. Leaf proofs are not zero-knowledge. They are only seen by the user who generated them and then included in the first aggregation layer as private inputs.

4. First-Layer Aggregation: The user batches leaf proofs that share the same `block_hash`, pads with dummy leaves if needed, shuffles the order, and produces a zero-knowledge wrapper proof that forwards grouped exits and nullifiers. A user may combine multiple deposits from distinct wormhole addresses (each derived from a different secret) into a single aggregated withdrawal. This step should be performed locally to preserve privacy.

5. Settlement: The blockchain verifies the aggregate proof, checks that the public `block_hash` matches the referenced block number, verifies that the fee parameter matches the configured rate, rejects reused nullifiers, and settles the exits.

5 Formal Specification and Interfaces

5.1 Leaf public inputs

The leaf proof exposes the following public fields:

- `asset_id`: Token identifier.
- `output_amount_1`, `output_amount_2`: Exit amounts (secondary is zero if unused).
- `fee_bps`: Fee parameter in basis points.
- `nullifier`: Replay-prevention tag derived from secret and counter.
- `exit_account_1`, `exit_account_2`: Destination accounts.
- `block_hash`: Commitment to block header fields.
- `block_number`: Height of the referenced block.

5.2 Leaf private witness

The private witness includes:

- `secret`: User secret from which nullifier and wormhole address are derived.
- `counter`: Per-deposit counter (`transfer_count`) used in nullifier derivation.
- `wormhole_account`: Derived wormhole address $\mathsf{WA}(s)$.
- `zk_tree_proof`: Merkle inclusion proof for the deposit transfer event in the block's ZK tree.
- `input_amount`: Deposited amount before fee deduction.
- Header fields (`parent_hash`, `state_root`, `extrinsics_root`, `zk_tree_root`, `digest`) used to recompute and bind the public `block_hash`.

5.3 Leaf relation

Let $x = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$ denote the public input tuple and let w denote the witness described above. The leaf relation $\mathcal{R}_{\text{leaf}}$ holds when there exists a witness w such that:

(C1) **Nullifier Correctness:** $n = \mathsf{Null}(s, c)$.

(C2) **Wormhole-Address Correctness:** $\alpha_{\text{wh}} = \mathsf{WA}(s)$.

(C3) **Authenticated Inclusion:** The ZK tree inclusion proof verifies that a transfer event to $\mathsf{WA}(s)$ with counter c and amount v_{in} exists under the tree root committed in the block header.

(C4) **Block Binding:** The inclusion proof is bound to the public block hash h_B and block number b via the header's `zk_tree_root` field and the circuit's block-hash commitment.

(C5) **Fee Constraint:** The disclosed exits satisfy

$$(v_1 + v_2) \cdot 10000 \leq v_{\text{in}} \cdot (10000 - f).$$

The inequality in (C5) allows users to withdraw less than their full balance. This provides a privacy benefit: by forfeiting some value, a user can choose output amounts that blend with more deposits, enlarging their anonymity set. The difference between the deposit and the sum of outputs (beyond fees) is effectively burned.

In the public input tuple and (C5), f denotes the fee in basis points. In the security and privacy analysis we overload f to mean the fractional rate $f/10^4$, so that (C5) reads $v_1 + v_2 \leq v_{\text{in}}(1 - f)$ and thresholds take the form $S/(1 - f)$.

5.4 On-chain verification

The proof system does not, by itself, settle a withdrawal. The blockchain must also:

- verify the referenced block exists in chain history
- check the fee parameter matches the configured rate
- reject previously-seen nullifiers
- credit exit accounts without increasing total supply

6 Recursive Aggregation Architecture

6.1 First-layer aggregation

The first aggregation layer is privacy-sensitive and should be executed locally. A batch of up to N leaf proofs is aggregated as follows:

- All real leaves must share the same `block_hash`, `asset_id`, and `fee_bps`.
- If fewer than N real leaves are available, the batch is padded with dummy leaves.
- The leaf order is shuffled before wrapping.
- Exits to identical accounts are grouped into a single summed output.
- Real nullifiers are forwarded; dummy nullifiers are replaced by $H(H(u))$ for prover-supplied random preimages u, preventing a malicious prover from choosing a dummy tag that collides with a real one.

A delegated first-layer service would observe raw child proofs before padding and shuffling, collapsing much of the anonymity set quantified in section 7.5.

6.2 Higher-layer aggregation

Higher layers aggregate already-wrapped proofs. Since they operate only on public outputs, they can be safely delegated, and zero-knowledge is not required in principle at these layers.

7 Security and Privacy Analysis

We formalize security via game-based definitions for the properties a wormhole system must satisfy. Let $\mathcal{B}$ denote a probabilistic polynomial-time adversary, H a hash function, and $\Pi = (\mathsf{Prove}, \mathsf{Verify})$ the proof system with knowledge soundness error ϵ_{ks}.

7.1 Deposit binding

Definition 1 (Deposit Binding Game)**.** The *deposit binding game* $\mathsf{Game}^{\mathcal{B}}_{\text{bind}}$ proceeds as follows:

1. $\mathcal{B}$ is given access to the blockchain state and may create arbitrary deposits.
2. $\mathcal{B}$ outputs a proof π and public inputs $x = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$.
3. $\mathcal{B}$ wins if $\mathsf{Verify}(\pi, x) = 1$ and there is *no* witness $w = (s, c, v_{\text{in}}, \ldots)$ satisfying $\mathcal{R}_{\text{leaf}}(x, w)$ whose authenticated deposit—the ZK tree leaf recording a transfer to $\mathsf{WA}(s)$ with counter c, asset a, and input amount v_{in}—is actually recorded in the ZK tree at block (h_B, b).

We say the system satisfies *deposit binding* if $\Pr[\mathsf{Game}^{\mathcal{B}}_{\text{bind}} = 1] \leq \epsilon_{\text{ks}} + \epsilon_{\text{coll}}$ for all PPT $\mathcal{B}$, where ϵ_{coll} is the collision probability of H.

Theorem 1 (Deposit Binding)**.** *The wormhole protocol satisfies deposit binding under knowledge soundness of* Π *and collision resistance of* H*.*

Proof. Suppose $\mathcal{B}$ wins. By knowledge soundness, the extractor outputs a witness w satisfying $\mathcal{R}_{\text{leaf}}(x, w)$—and hence conditions (C1)–(C5)—except with probability ϵ_{ks}. Because $\mathcal{B}$ wins, no satisfying witness has its deposit recorded in the tree; in particular, the extracted w's deposit tuple $(\mathsf{WA}(s), c, a, v_{\text{in}})$ is absent from the ZK tree at block (h_B, b). Yet condition (C3) requires w to include a ZK tree inclusion path for exactly this tuple that verifies against the root committed in (h_B, b) per (C4). A verifying path for an absent leaf requires a collision in the tree's Poseidon hash (probability ϵ_{coll}). The blockchain's validation that (h_B, b) refers to a real block ensures the committed root is the correct one for that height. □

7.2 One-time withdrawal

Definition 2 (Double-Spend Game)**.** The *double-spend game* $\mathsf{Game}^{\mathcal{B}}_{\text{double}}$ proceeds as follows:

1. $\mathcal{B}$ creates a single deposit of amount v to address $\mathsf{WA}(s)$ for adversarially chosen s.
2. $\mathcal{B}$ outputs two proofs (π_1, x_1) and (π_2, x_2) with nullifiers $n_1 \neq n_2$.
3. $\mathcal{B}$ wins if both proofs verify and both claim outputs from the same deposit.

We say the system satisfies *one-time withdrawal* if $\Pr[\mathsf{Game}^{\mathcal{B}}_{\text{double}} = 1] \leq \epsilon_{\text{ks}} + \epsilon_{\text{coll}}$, where ϵ_{coll} is the collision probability of H.

Theorem 2 (One-Time Withdrawal)**.** *The wormhole protocol satisfies one-time withdrawal under knowledge soundness of* Π *and collision resistance of* H*.*

Proof. Suppose $\mathcal{B}$ wins. By knowledge soundness, there exist witnesses w_1, w_2 with secrets s_1, s_2 and counters c_1, c_2 such that both proofs reference the same deposit. Since both reference the same deposit address, $\mathsf{WA}(s_1) = \mathsf{WA}(s_2)$, which by collision resistance implies $s_1 = s_2$. The counter c is part of the ZK tree leaf and verified by the inclusion proof; since both proofs reference the same deposit, they must use the same counter $c_1 = c_2$. But then $n_1 = \mathsf{Null}(s_1, c_1) = \mathsf{Null}(s_2, c_2) = n_2$, contradicting $n_1 \neq n_2$. Thus $\mathcal{B}$ can only win by breaking knowledge soundness or finding a collision. □

Since the blockchain maintains a nullifier set and rejects duplicates, each deposit can be withdrawn at most once.

7.3 Spend-path exclusivity

Definition 3 (Signature Forgery Game)**.** The *signature forgery game* $\mathsf{Game}^{\mathcal{B}}_{\text{sig}}$ proceeds as follows:

1. Challenger samples $s \leftarrow \{0,1\}^{256}$ and publishes $\mathsf{WA}(s)$.
2. $\mathcal{B}$ outputs a keypair (sk, pk) and a signature σ on an arbitrary message m.
3. $\mathcal{B}$ wins if $H(pk) = \mathsf{WA}(s)$ and $\mathsf{SigVerify}(pk, m, \sigma) = 1$.

We say the system satisfies *spend-path exclusivity* if $\Pr[\mathsf{Game}^{\mathcal{B}}_{\text{sig}} = 1] \leq \epsilon_{\text{pre}}$, where ϵ_{pre} is the preimage resistance of H.

Theorem 3 (Spend-Path Exclusivity)**.** *The wormhole protocol satisfies spend-path exclusivity under preimage resistance of H.*

Proof. To win, $\mathcal{B}$ must find pk such that $H(pk) = \mathsf{WA}(s) = H(H(\mathsf{salt}_{\mathsf{wh}} \parallel s))$. This requires either:

1. finding s' such that $H(\mathsf{salt}_{\mathsf{wh}} \parallel s')$ equals a public key $\mathcal{B}$ controls, or
2. finding pk such that $H(pk)$ equals the target address.

Both are preimage attacks. For post-quantum schemes where $|pk| > |H|$ (e.g., ML-DSA), case (1) is impossible since a hash output cannot equal a larger public key. In either case, success probability is bounded by ϵ_{pre}. □

7.4 Value conservation

Definition 4 (Inflation Game)**.** The *inflation game* $\mathsf{Game}^{\mathcal{B}}_{\text{infl}}$ proceeds as follows:

1. $\mathcal{B}$ creates deposits totaling value V.
2. $\mathcal{B}$ submits a sequence of valid proofs resulting in withdrawals totaling value W.
3. $\mathcal{B}$ wins if $W > V \cdot (1 - f)$, where f is the fee rate.

Theorem 4 (Value Conservation)**.** *The wormhole protocol satisfies value conservation: no adversary can win* $\mathsf{Game}_{\text{infl}}$ *except with negligible probability.*

Proof. Value conservation follows from three properties:

- Each leaf proof enforces (C5): outputs satisfy $(v_1 + v_2) \cdot 10000 \leq v_{\text{in}} \cdot (10000 - f)$.
- Aggregation preserves output sums; the wrapper only groups outputs to identical destinations.
- One-time withdrawal (theorem 2) prevents double-counting any deposit.

Together with the blockchain's nullifier check, total settled value never exceeds total deposited value minus fees. □

7.5 Anonymity sets

An adversary observing a first-layer aggregate sees N nullifiers (some real, some dummies), grouped exits, and the public metadata (b, a, f). Deposit unlinkability is analyzed by enumerating plausible funding subsets in $\mathcal{D}(b, a)$; nullifier indistinguishability is a separate property that hides padding and batch composition. Throughout this subsection, H denotes the Poseidon2 hash used in-circuit, modeled as a random oracle with 256-bit outputs.

Dummy nullifiers. Real nullifiers are $\mathsf{Null}(s, c) = H(H(\mathsf{salt}_{\mathsf{null}} \parallel s \parallel c))$ as in section 5. Dummy padding replaces each dummy slot with

$$\mathsf{DNull}(u) = H(H(u)),$$

where the prover samples $u \leftarrow \{0,1\}^{256}$ and the aggregation circuit recomputes the outer hash (matching the implementation).

Definition 5 (Nullifier Indistinguishability Game)**.** The *nullifier indistinguishability game* $\mathsf{Game}^{\mathcal{B}}_{\mathrm{null}}$ proceeds as follows:

1. Challenger samples secret $s \leftarrow \{0,1\}^{256}$, counter $c \in \mathbb{N}$, and dummy preimage $u \leftarrow \{0,1\}^{256}$.
2. Challenger computes $n_0 = \mathsf{Null}(s, c)$ and $n_1 = \mathsf{DNull}(u)$.
3. Challenger flips $b \leftarrow \{0,1\}$ and sends n_b to $\mathcal{B}$.
4. $\mathcal{B}$ outputs guess b'.
5. $\mathcal{B}$ wins if $b' = b$.

We say nullifiers are *indistinguishable* if for all PPT $\mathcal{B}$ making at most q_H queries to the random oracle H,

$$\left|\Pr[\mathsf{Game}^{\mathcal{B}}_{\mathrm{null}} = 1] - \tfrac{1}{2}\right| \leq \epsilon_{\mathrm{ro}}(q_H),$$

where $\epsilon_{\mathrm{ro}}(q_H)$ is negligible in the hash output length $\lambda = 256$ for fixed q_H.

Theorem 5 (Nullifier Indistinguishability)**.** *In the random oracle model, real and dummy nullifiers are computationally indistinguishable:* $\epsilon_{\mathrm{ro}}(q_H) = O(q_H^2/2^\lambda)$.

Proof. Write $p = \mathsf{salt}_{\mathsf{null}} \parallel s \parallel c$ and let $x_0 = H(p)$, $x_1 = H(u)$, so $n_0 = H(x_0)$ and $n_1 = H(x_1)$. The fixed salt and public counter c do not weaken the analysis: s is uniform and independent of u, so p is unpredictable to $\mathcal{B}$ before the challenge unless it queries $H(p)$.

In the random oracle model, x_0 and x_1 are uniform and independent on their first queries, and so are n_0 and n_1 unless $\mathcal{B}$ has previously queried $H(x_0)$ or $H(x_1)$. Because s is uniform and unknown to $\mathcal{B}$, the real preimage p is unpredictable (even though $\mathsf{salt}_{\mathsf{null}}$ and c are fixed or public), and the dummy preimage u is uniform and independent. The probability that any of $\mathcal{B}$'s q_H queries hits p, u, x_0, or x_1 before the challenge is therefore at most $O(q_H^2/2^\lambda)$ by a standard random-oracle union bound. Conditioned on no such query, n_0 and n_1 are identically distributed and independent of b, so $\Pr[b' = b] = 1/2$. □

Instantiating H with Poseidon2 is heuristic, as is standard for random-oracle-based designs: collision and preimage resistance alone do not imply the pseudorandomness used above, but no structural property of Poseidon2 is known that would distinguish double-hash outputs of unpredictable inputs.

Definition 6 (Aggregate view)**.** A first-layer aggregate Y exposes block number b, asset identifier a, fee rate f, batch capacity N, a multiset of N nullifiers, and grouped exits $\{(\alpha_j, e_j)\}_{j=1}^{E}$ with total $S = \sum_{j=1}^{E} e_j$.

Definition 7 (Plausible deposit subsets)**.** Let $\mathcal{D}(b, a)$ be the unspent wormhole deposits recorded in the ZK tree at block b with asset a, with amounts $a_1, \ldots, a_M$. A subset $I \subseteq \{1, \ldots, M\}$ is *plausible* for Y if:

(i) **Batch size:** $|I| \leq N$.

(ii) **Output slots:** $E \leq 2|I|$, since each authorization exposes at most two outputs before grouping.

(iii) **Volume:** $\sum_{i \in I} a_i \geq S/(1-f)$ (equivalently, $\sum_{i \in I} a_i(1-f) \geq S$).

The *anonymity set* is

$$\mathcal{A}(Y) = \{I \subseteq \{1, \ldots, M\} \mid I \text{ is plausible for } Y\}.$$

Each authorization can route its two outputs arbitrarily among the observed exit accounts, subject only to its per-deposit cap $a_i(1-f)$; given (ii)–(iii), such a routing always exists for any split $\{e_j\}$ with $\sum_j e_j = S$. The public per-account amounts therefore do not further restrict $\mathcal{A}(Y)$ beyond the total S. The main refinements over a volume-only definition are the deposit pool $\mathcal{D}(b, a)$, the batch cap N, and the slot bound $E \leq 2|I|$. Counting $|\mathcal{A}(Y)|$ is an instance of #SubsetSum with threshold $T = S/(1-f)$, plus $E \leq 2|I|$. The definition depends on exits and metadata only; published nullifiers do not identify deposits without inverting $\mathsf{Null}(s, c)$.

Theorem 6 (Transaction Unlinkability)**.** *Let Y be a first-layer aggregate with $|\mathcal{A}(Y)| = k \geq 1$. Assume the true funding subset is drawn uniformly from $\mathcal{A}(Y)$. Then no PPT adversary can identify the true subset with probability greater than $1/k$.*

Proof. An adversary attempting to identify the funding deposits observes Y and the deposit pool $\mathcal{D}(b, a)$, computes $\mathcal{A}(Y)$, and guesses which subset was used. Any guess outside $\mathcal{A}(Y)$ is inconsistent with the public exit total S, the slot bound, the batch cap, or the block/asset filter, and can be ruled out without guessing.

Under the uniform prior over $\mathcal{A}(Y)$, every plausible subset is equally likely, so success probability is at most $1/k$. Any strategy exceeding $1/k$ must use information outside this model (e.g., side channels, deposit clustering, or a non-uniform prior over user behavior). Nullifiers are not needed for this attack: each published nullifier is a one-way tag $\mathsf{Null}(s, c)$ that does not reveal which deposit funded the withdrawal. □

Role of nullifier indistinguishability. Theorem 5 addresses a different question: whether an adversary can tell real nullifiers from dummy padding, or infer how many real authorizations appear in a batch. This hides batch composition but does not change the deposit-unlinkability analysis above, which already treats only $\mathcal{A}(Y)$. Without indistinguishability, an adversary who learns the number of real authorizations could restrict attention to subsets of that exact size, potentially shrinking the effective candidate set below k.

One structural caveat applies in the implemented output layout: exit slots are positionally aligned with nullifier slots (slots $2i$ and $2i{+}1$ belong to authorization i), and each distinct exit account retains its grouped sum at one first-occurrence slot. An observer therefore learns that the authorizations owning non-zero exit slots are real, marking those specific nullifiers as real and yielding a lower bound of up to E on the real count (versus $\lceil E/2 \rceil$ from the exits alone). Real authorizations whose outputs were grouped into earlier

slots remain indistinguishable from dummies, and no nullifier-to-deposit linkage arises, so the deposit-unlinkability bound of theorem 6 is unaffected.

Because the proof system enforces an inequality rather than exact equality (condition C5), the adversary must consider any subset whose deposits can fund at least S after fees. Users may also withdraw less than their full balance, enlarging the volume-feasible region at the cost of burned value. The slot bound $E \leq 2|I|$ is the main structural constraint beyond volume: a subset of size $|I|$ cannot explain more than $2|I|$ distinct exit accounts.

Complexity of counting plausible subsets: The decision version of subset sum is NP-complete [GJ79]. The counting version #SubsetSum is #P-complete [Val79]: the standard reduction establishing NP-completeness of subset sum is parsimonious, so it transfers to the counting class. In the special case $E = 1$ and $N = M$, condition (ii) is vacuous and $\mathcal{A}(Y)$ reduces to #SubsetSum with threshold $T = S/(1 - f)$.

Theorem 7 (Hardness of Anonymity Set Computation)**.** *Computing $|\mathcal{A}(Y)|$ is #P-hard.*

Proof. We reduce from #SubsetSum. Given an instance $(a_1, \ldots, a_M, T)$, construct an aggregate view with $E = 1$, exit (α_1, e_1) where $e_1 = T(1 - f)$, batch cap $N = M$, and deposit pool $\mathcal{D}(b, a)$ containing the same amounts. For any $I \subseteq \{1, \ldots, M\}$, condition (iii) holds iff $\sum_{i \in I} a_i \geq T$; condition (ii) holds because $E = 1 \leq 2|I|$ for every non-empty I. Thus $|\mathcal{A}(Y)|$ equals the number of subsets summing to at least T. Given oracle access to $|\mathcal{A}(Y)|$, two calls with thresholds T and $T + 1$ recover #SubsetSum exactly as in the standard threshold-to-exact conversion. Hence computing $|\mathcal{A}(Y)|$ is #P-hard. □

Burn Cost and Bounded Rationality: A rational adversary may assume users are unwilling to burn significant value for privacy. Under bounded rationality, the effective anonymity set shrinks to subsets whose total deposit volume is at most $(1 + \delta) \cdot S/(1 - f)$ for some small δ (e.g., 1%)—that is, at most slightly above the minimum volume required by condition (iii)—in addition to the slot bound above.

7.6 Privacy leakage

Output amounts, exit addresses, nullifiers, and timing are public. The unlinkability bound in theorem 6 assumes a uniform prior over plausible deposit subsets and analyzes the exit-driven candidate set $\mathcal{A}(Y)$; amount patterns, address reuse, deposit clustering, and timing correlation can shrink the effective set further. Nullifier indistinguishability (theorem 5) hides dummy padding but is orthogonal to deposit identification. First-layer padding enlarges the anonymity set, but this is plausible deniability, not full shielding.

8 Evaluation

8.1 Methodology

Benchmarks were executed on an Apple M2 Max with 12 CPU cores and 32 GB of RAM. Circuit artifacts were pre-generated; reported times measure only proving and verification. Prove times include witness assignment and proof generation. Verify times measure verification of a pre-generated proof.

All first-layer measurements use repeated identical dummy leaf proofs with the same `block_hash`. Second-layer benchmarks use repeated first-layer proofs.

8.2 Leaf proofs

Individual leaf proof generation takes approximately 20 ms, with verification in 1.6 ms. These times are dominated by proving; witness assignment is negligible.

Table 1: First-layer aggregation times for N leaf proofs.

N leaf proofs	Prove (s)	Verify (ms)
2	1.55	2.75
4	2.81	2.88
8	5.39	3.02
16	10.74	3.16
32	21.71	3.55

Table 2: Second-layer aggregation times. Each first-layer proof aggregates 8 leaf proofs.

M first-layer proofs	Total leaves	Prove (s)	Verify (ms)
2	16	0.92	2.72
4	32	1.88	2.86
8	64	3.84	3.03
16	128	7.97	3.24
32	256	16.69	3.74

8.3 First-layer aggregation

Proving time scales with batch size, while verification remains in the millisecond range across all tested configurations.

8.4 Second-layer aggregation

Second-layer verification time grows slowly with batch size (2.72 ms at 16 leaves to 3.74 ms at 256 leaves), remaining in the low millisecond range and demonstrating the benefit of recursive composition.

8.5 On-chain footprint

The final aggregate proof is constant-size regardless of the number of authorizations. The marginal cost consists only of the public outputs that must be published. Each authorization contributes two exit slots, each comprising an exit address (32 bytes) and a quantized amount (8 bytes, one field element), plus one nullifier (32 bytes): 112 bytes per authorization, or **56 bytes per settled output**. For comparison, an ML-DSA-87 signature alone is 4,627 bytes—over 80× larger—and a full post-quantum transaction with its 2,592-byte public key would exceed 7 KB, a >125× reduction in per-output data, while providing privacy guarantees that signatures cannot offer.

9 Related Work

Wormholes and Privacy Pools: EIP-7503 proposes proof-of-burn reminting via secret-derived addresses [KBK+23]. Privacy Pools adds compliance-aware privacy sets [BIN+23]. Our work extends this line with recursive proofs that authenticate deposits via a consensus-maintained ZK tree.

Shielded Systems: Zcash provides strong privacy through shielded notes; Tornado Cash popularized mixer-style privacy on Ethereum [HBH+23, PSS19]. Our approach offers lighter-weight private settlement over visible transfer events, authenticated via a consensus-maintained ZK tree rather than encrypted note commitments.

Recursive Proofs: Halo and Plonky2 are standard recursive proof systems [BGH19, Pol22]. Plonky3 provides Poseidon2 primitives with a separate recursion track [Plo24]. We use Plonky2 with Poseidon2 for recursive composition.

Hash-Based Authorization: The Ziggy signature scheme [GR20] uses a ZK-STARK to prove knowledge of a hash preimage, replacing lattice-based signatures with hash-based proofs.

Post-Quantum Signatures: Adoption of post-quantum signatures like ML-DSA and SLH-DSA raises questions about communication and verification cost [NIST24]. Our approach compresses authorization into ZK proofs rather than publishing large signatures.

10 Conclusion

We presented a hash-based authorization scheme where users commit funds to addresses derived from secret preimages and later claim them by proving knowledge of the preimage in zero knowledge. This approach sidesteps post-quantum signature verification entirely, requiring only hash evaluations and membership proofs inside the circuit. Recursive aggregation via Plonky2/FRI amortizes verification cost, while first-layer dummy padding enlarges anonymity sets against balance-linking attacks. We formalized the withdrawal relation, proved six security properties via game-based definitions, showed that computing anonymity set size is #P-hard, and demonstrated practical performance on commodity hardware.

Acknowledgments

We thank the anonymous reviewers for their helpful feedback.

References

[BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP)*, 2018. https://eprint.iacr.org/2017/568

[BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021, 2019. https://eprint.iacr.org/2019/1021

[BIN+23] Vitalik Buterin, Jacob Illum, Matthias Nadler, Fabian Schär, and Ameen Soleimani. Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium. SSRN, 2023. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4563364

[GR20] Lior Gold and Michael Riabzev. Ziggy: A Zero-Knowledge Signature Scheme Based on STARKs. StarkWare, 2020. https://github.com/starkware-libs/ethSTARK

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. Cryptology ePrint Archive, Report 2023/323, 2023. https://eprint.iacr.org/2023/323

[HBH+23] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash Protocol Specification. Protocol specification, 2023. https://zips.z.cash/protocol/protocol.pdf

[KBK+23] Keyvan Kambakhsh, Hamid Bateni, Amir Kahoori, and Nobitex Labs. EIP-7503: Zero-Knowledge Wormholes. Ethereum Improvement Proposal, 2023. https://eips.ethereum.org/EIPS/eip-7503

[NIST24] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard (FIPS 204). Federal Information Processing Standards Publication 204, 2024. https://csrc.nist.gov/pubs/fips/204/final

[Pol22] Polygon Zero Team. Plonky2: Fast Recursive Arguments with PLONK and FRI. Software repository, 2022. https://github.com/0xPolygonZero/plonky2

[PSS19] Alexey Pertsev, Roman Semenov, and Roman Storm. Tornado Cash Privacy Solution. Project documentation, 2019. https://tornado.cash

[Plo24] Plonky3 Contributors. Plonky3. Software repository, 2024. https://github.com/Plonky3/Plonky3

[GJ79] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[Val79] Leslie G. Valiant. The Complexity of Enumeration and Reliability Problems. *SIAM Journal on Computing*, 8(3):410–421, 1979.

[Sho97] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.